Trabalho Prático 3 - Busca de Passagens Aéreas

João Gabriel Guimarães Alves Vaz
Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil
Matrícula: 2023099166
jgvaz@ufmg.br

2024

1 Introdução

O trabalho proposto tratava de um problema de busca de passagens aéreas em uma agência fictícia. Foi entregue um arquivo contendo informações sobre as passagens aéreas disponíveis (local de origem e de destino, preço, número de assentos disponíveis, data e hora de chegada e de partida), além de uma lista de consultas (que especificavam restrições sobre as propriedades das passagens aéreas). O objetivo do trabalho era simular um sistema de busca de forma a, ao final, imprimir os "melhores resultados" para uma determinada consulta (ou seja, as n passagens que melhor "ranqueavam" nos atributos dados, considerando um critério de ordenação também dado). A situação-problema, junto da especificação, deixa evidente a necessidade da implementação de várias estruturas de dados de forma a manter registro das passagens, das consultas, além de garantir uma pesquisa eficiente (não exaustiva); não só isso, o problema também exigiu uso de algoritmos de ordenação.

Abaixo está um resumo do que será abordado em cada uma das seções deste documento.

- Na seção 1, há uma introdução ao tema, o contexto e o sumário.
- Na seção 2, há o método de implementação.
- Na seção 3, há a análise de complexidade dos procedimentos implementados.
- Na seção 4, há uma discussão das estratégias de robustez utilizadas.
- Na seção 5, há uma análise experimental.
- Na seção 6, há a conclusão desta documentação.
- Na seção 7, há a bibliografia.

2 Método

O programa foi desenvolvido na linguagem C++, compilada pelo compilador G++ da GNU Compiler Collection.

2.1 Tipos abstratos de dados

No total, foram implementados 5 tipos abstratos de dados para o desenvolvimento do programa. O principal deles é o TAD Passagem, que armazena as informações das passagens aéreas, funções getters, overload de operadores (para a comparação na hora de ordenar os voos), e métodos auxiliares de leitura, preenchimento de informações, cálculo de duração e a ordenação propriamente dita.

O segundo TAD mais importante é o TAD Consulta, junto do TAD Cláusula. Esses dois são responsáveis pelo parsing e armazenamento das informações das consultas realizadas em cima dos voos. Uma Consulta é uma expressão conjuntiva formada por cláusulas. Cada Cláusula é uma expressão relacional formada por dois campos (especificamente, "membro" - que faz referência a uma variável da Passagem, por exemplo, duração, ou destino - e "valor" - que é o valor daquele campo assumido por uma passagem) e um operador de comparação (>=, <= ou ==). Dessa forma, uma Consulta é um vetor de Cláusulas, e é avaliado de forma conjuntiva (isto é, todas as cláusulas precisam retornar verdadeiro para a consulta retornar verdadeiro). Uma Consulta ainda possui métodos auxiliares de leitura e de escrita, além de duas funções especiais: uma que avalia uma passagem (ou seja, retorna um booleano que indica se a passagem dada satisfaz ou não a consulta) e outra que retorna uma lista das passagens que atendem aquela determinada consulta.

Além dos 3 TADs citados acima, têm-se ainda duas estruturas auxiliares de dados que são essenciais para o fluxo do programa. São elas: Árvore e Lista. A primeira é usada para tornar a busca de passagens aéreas mais eficiente: utilizar árvores torna a busca de passagens que atendam às consultas sublinear, como será descrito mais para frente neste documento. Já a segunda é utilizada mais como um vetor dinâmico (sem tamanho fixo) do que como uma lista de inserção e remoção em qualquer posição. A Lista é usada para armazenar ponteiros de Passagens que atendam às consultas (dessa forma, fica fácil intercalar as listas ao final do processo).

2.2 Módulos

Neste trabalho, os módulos usados foram as implementações dos arquivos hpp de declarações de classes e métodos. Ou seja, as únicas funções existentes (além da main) são os métodos das classes implementadas. Nesse sentido, têm-se métodos getters e setters para variáveis de Passagem, Consulta, Cláusula, Árvore e Lista. Além disso, têm-se as funções estáticas auxiliares que verificam a validade dos dados e de preenchimento de informações das classes. Também têm-se uma função de ordenação, que utiliza princípios da ordenação indireta desenvolvida no TP1.

2.3 Funcionamento do programa

O objetivo desta subseção é detalhar não as escolhas técnicas de implementação, mas sim o fluxo, em alto nível, do programa desenvolvido. Como o objetivo era simular a busca por passagens aéreas (em complexidade sublinear), fiz uso de árvores para armazenar as passagens, aproveitando o fato de que a pesquisa em árvores balanceadas é $O(logn)$. Essa é a "chave" da implementação, o ponto principal. As outras estruturas desenvolvidas são apenas auxiliares para armazenar informações de voos, consultas e etc.

Detalhando com mais profundidade o fluxo lógico do programa:

- De início, o arquivo de entrada é aberto para a leitura das informações.
- Após a abertura, são declarados e preenchidos os vetores em heap que armazenam as informações de passagens e de consultas. Isso é feito, respectivamente, pelas funções estáticas lerPassagens e lerConsultas. A primeira simplesmente percorre todas as posições do vetor de passagens e atribui corretamente as informações lidas do arquivo (origem, destino, tempo de voo, etc). A segunda faz o mesmo, mas com o vetor de consultas: percorre as consultas do arquivo e atribui as informações aos elementos do vetor e às variáveis internas da classe de consultas, garantindo, dessa forma, o fluxo correto de execução do programa.
- Agora, são criadas 8 árvores (uma para cada categoria das passagens) e preenchidas pela função também estática preencheArvores. Essa função percorre uma por uma, todas as passagens e, dentro desse laço de repetição, percorre os 8 atributos que uma passagem aérea possui (origem, destino, etc), inserindo, em cada uma das 8 árvores, a informação correspondente de forma correta (isto é, adicionando aquela passagem à lista de passagens do nó caso já exista naquela árvore com aquele valor, ou criando um nó novo e adicionando-o à árvore, caso contrário).
- Nesse ponto, há o único laço de repetição da função main. Essa estrutura percorre o vetor de consultas e, para cada consulta, encontra as passagens que a satisfazem, ordena-as com base no trigrama dado e imprime-as na tela.

- Em termos de alto nível, o algoritmo de encontrar as passagens aéreas que atendem a uma determinada consulta é o seguinte: para cada cláusula da consulta, encontrar as passagens que a satisfazem. Após isso, juntar essas listas de passagens (certificando-se de colocar na lista final apenas aquelas passagens que satisfazem todas as cláusulas simultaneamente).
- Por sua vez, o algoritmo de encontrar as passagens que atendem a uma cláusula é o que se segue: dada uma passagem aérea e uma cláusula (ex, $dur \leq 500$), percorrer a árvore correspondente (nesse caso, a árvore cujos nós representam a duração dos voos) de forma recursiva para encontrar aqueles nós que atendem à cláusula (ex, aqueles nós cuja duração é menor ou igual a 500). Isso é feito utilizando as informações da cláusula dadas na string da consulta, armazenadas de forma compreensível pela função `Consulta::parse`. O caminhamento na árvore para encontrar a satisfação da cláusula aproveita a estrutura da árvore de pesquisa AVL, de forma a procurar recursivamente apenas nos nós "corretos".
- Por fim, há as instruções de desalocação de memória, como os comandos *delete*[] e o método *close*() do arquivo de entrada.

Vale notar que cada um dos blocos descritos acima estão envelopados em estruturas de try/catch. Isso foi feito como estratégia de robustez, para tentar evitar ao máximo erros durante a execução do programa.

2.4 Instruções de compilação

Para executar o programa, basta chamar o utilitário *make* - que vai compilar o executável e colocá-lo em *./bin/tp*3.*out* - e chamar o programa, dessa forma:

```
make
./bin/tp3.out <entrada>
```

3 Análise de complexidade

Tendo em vista que há muitos arquivos de código, cada um com suas funções específicas, considerei que o ideal era uma análise módulo por módulo da complexidade de cada função. Portanto, o que se segue são parágrafos enumerando, descrevendo e analisando as complexidades temporais e espaciais das diversas funções das várias classes implementadas na confecção do TP.

3.1 Tempo

3.1.1 Árvore

- `Árvore::insere`: o TAD árvore implementa uma árvore de pesquisa AVL. Nesse sentido, como a árvore é mantida sempre balanceada, a inserção é $O(logn)$, sendo n o número de nós.
- `Árvore::remove`: pelo mesmo motivo explicado acima, a remoção de AVL é $O(logn)$.
- `Árvore::find`: pela mesma razão descrita acima, a busca por um determinado nó numa árvore AVL é $O(logn)$.
- `Árvore::findMin`: a função findMin procura o menor nó em uma subárvore, acessando os nós da esquerda recursivamente até encontrar um nó folha. Nesse sentido, por um raciocínio similar ao que explica o motivo da busca ser logarítmica em árvores AVL, a busca por um nó mínimo também é $O(logn)$, sendo n o número de nós da subárvore em questão.
- `Árvore::preencherÁrvores`: essa função é responsável pelo preenchimento correto das informações no vetor dinâmico de árvores. Isso é feito por meio de laços de repetição, que iteram sobre todas as passagens aéreas registradas e, para cada uma dessas, iteram 8 vezes (há 8 atributos que caracterizam árvores em cada passagem) e, em cada uma dessas 8 iterações, executam as funções `find` e `insere` do tipo árvore; adicionando a passagem na lista de passagens do nó caso já exista na árvore com aquele valor, ou criando um novo nó e adicionando-o à árvore, caso contrário. Sendo n o número de passagens, a complexidade dessa função é $O(n * 8 * 2logn) = O(nlogn)$.

- As demais funções do tipo Árvore são $O(1)$. Isso porque elas são responsáveis por realizar rotações, recuperar a altura de um nó e calcular o fator de balanceamento. Tendo em vista que a altura de cada nó é armazenada a priori no tipo abstrato de dados auxiliar Node, esses cálculos não dependem do número de nós da árvore. Todos esses procedimentos são feitos em um número constante de operações.

3.1.2 Consulta e Cláusula

- `Consulta::avaliarPassagem`: essa função percorre as cláusulas da consulta e verifica se a passagem em questão atende a todas elas simultaneamente. Dessa forma, é $O(k)$ sendo k o número de cláusulas.

- `Consulta::acharPassagensQueSatisfacam`: essa função retorna uma lista das passagens que satisfazem a uma consulta. Esse procedimento percorre a lista de cláusulas de uma consulta e testa a validade de cada uma delas em todas as passagens das árvores de passagens aéreas. A função que encontra as passagens que satisfazem a uma cláusula específica é `acharPassagensQueSatisfacam` (método da classe Cláusula, não da Consulta). Essa função faz uso das estruturas ramificadas e balanceadas das árvores AVL, tornando-a $O(logn)$ (pelos motivos explicados na subseção anterior, com n sendo o número de passagens aéreas). Dessa forma, a função de achar passagens que satisfazem a uma cláusula é $O(klogn) = O(logn)$, sendo k o número de cláusulas da consulta. Isso porque, para cada uma das k cláusulas, há a invocação de uma função logarítmica.

- `Consulta::lerConsultas`: essa função estática preenche as informações das consultas. Sendo k o número de consultas do arquivo de entrada, a função percorre cada um dos k elementos do vetor de consultas e atribui a ele as informações daquela consulta. Nesse sentido, esse método é $O(k)$.

- As demais funções são construtores, destruidores, getters, setters e uma função de parsing. Essa última realiza a interpretação de uma consulta, transformando a string que a representa (por exemplo, $(dur \leq 500)\&\&(prc == 899)$) em um conjunto de elementos Cláusula, atribuindo corretamente as informações da string aos respectivos campos do tipo abstrato de dados Cláusula. Ainda que essa função seja relevante, em nível de complexidade temporal, ela executa um número fixo de instruções, da mesma forma que as demais citadas nesse inciso. Assim, essas funções restantes são todas $O(1)$.

- A respeito do TAD Cláusula, não há muito a ser dito. A única função expressiva é a de achar passagens que satisfazem a cláusula em questão (`acharPassagensQueSatisfacam`), cuja complexidade, como descrito acima, é $O(logn)$, sendo n o número de passagens. As demais funções, embora possam possuir laços de repetição, não dependem do número de passagens aéreas; e podem ser, dessa forma, consideradas constantes em tempo de execução ($O(1)$).

3.1.3 Passagem

- `Passagem::lerPassagens`: essa função percorre todas as passagens aéreas apenas uma vez, atribuindo-as ao vetor de passagens aéreas. A finalidade disso é armazenar esses dados para que eles sejam posteriormente consumidos pelas outras funções que compõem o programa. Sendo composta de um laço linear de repetição em cima da quantidade de passagens aéreas (n), a função é $O(n)$.

- `Passagem::ordena`: esse método implementa um quicksort de ordenação indireta. Ou seja, recebe um vetor de ponteiros de passagens aéreas e o ordena de forma indireta, como visto no TP1. Sendo um quicksort, sua complexidade temporal é $O(nlogn)$ em cima do número de passagens. No pior caso, pode ser $O(n^2)$.

- As demais funções realizam um número fixo de operações, sendo, portanto, $O(1)$ em tempo. Especificamente, há 8 getters, 1 setter, 5 sobrecargas de operadores (utilizados no quicksort) e 7 métodos auxiliares: cálculo de duração do voo (extrai informações da string de partida e chegada para calcular o tempo em segundos), determinação se um valor é ou não numérico (pois tudo é armazenado como string; dados numéricos são strings com caracteres apenas numéricos ou ponto), e 5 funções de comparação entre strings (que convertem para double previamente caso a string represente um número real). Como mencionado, ainda que algumas dessas funções apresentem laços de repetição, todas executam um número fixo de instruções que não depende da quantidade de passagens aéreas na entrada. Logo, $O(1)$.

3.1.4 Lista e Célula

Esses tipos abstratos de dados auxiliares foram implementados de forma genérica e dinâmica (utilizando ponteiros em vez de vetores fixos) seguindo as instruções do material disponibilizado no Moodle da disciplina (consta nas referências bibliográficas). Note que, nessa parte, n se refere ao número de elementos da lista. A complexidade temporal é a que se segue, método por método:

- `Lista::InsereInicio, Lista::RemoveInicio`: $O(1)$, pois não é necessário "arredar" e, assim, apenas um número fixo de instruções é realizado.
- `Lista::InserePosicao, Lista::RemovePosicao, Lista::InsereFinal, Lista::RemoveFinal`: $O(n)$, pois percorre, no pior caso, todos os elementos da lista.
- `Lista::Limpa, Lista::Contem, Lista::Imprime`: $O(n)$ pela mesma razão explicada acima.
- `Lista::Juntar`: essa função junta disjuntamente duas listas. Sendo k o número de elementos da lista "a ser juntada", essa função é $O(kn) = O(n)$, pois percorre todos os elementos da lista secundária e verifica, utilizando a função Contem, se eles já existem na lista principal.
- `Lista::Posiciona`: esse método recebe um inteiro e posiciona o "cursor" da lista naquele ponto. Sendo k o inteiro recebido, esse método é $O(k)$, pois percorre todos os elementos da lista até aquele ponto. O pior caso dessa função é $O(n)$.
- `Lista::ToArray`: esse método retorna um array em heap com os elementos da lista. Ele faz isso alocando tal vetor e percorrendo a lista elemento a elemento atribuindo o elemento à posição contígua do vetor. Sendo assim, sua complexidade é $O(n)$.
- As demais funções são getters ou setters que executam um número fixo de instruções (independentemente do tamanho da lista). Dessa forma, as demais funções são $O(1)$.

3.1.5 Função main

Por fim, eis a análise de complexidade temporal da função principal do programa. Essa é a função que chama todas as outras funções. Essa função é responsável pelo preenchimento correto das informações; pela busca nas árvores; pela ordenação; e pela impressão. Nesse sentido, a complexidade final da função main é a maior das suas "subcomplexidades" (isto é, das complexidades dos trechos de código que a compõem). Nesse sentido, as subcomplexidades são: $O(n)$ para preencher o vetor de passagens aéreas (n o número de passagens), $O(k)$ para preencher o vetor de consultas (sendo k o número de consultas), $O(nlogn)$ para preencher o vetor de árvores e $O(knlogn)$ para ordenar e imprimir as passagens que atendam a uma determinada cláusula. Ou seja, a complexidade final da função main é $O(nlogn)$.

3.2 Espaço

3.2.1 Árvore

A inserção, busca e remoção de elementos em uma árvore AVL tem complexidade espacial $O(n)$, sendo n o número de nós da árvore.

3.2.2 Consulta e Cláusula

Assim como foi com a árvore, a análise de complexidade espacial do TAD Consulta é bem mais simples do que a temporal. Ainda que tenha métodos com laços de repetição e às vezes recursão, a maioria das funções dessa classe toma uma quantidade fixa de espaço, sendo, portanto, $O(1)$. A grande exceção é a função `Consulta::acharPassagensQueSatisfacam`. Essa função encontra, para uma determinada consulta, as k passagens que a satisfazem (ou seja, que satisfazem todas suas cláusulas simultaneamente). Dessa forma, devido à necessidade de alocação de listas, o método se torna linear no tamanho de passagens "satisfatórias", isto é, $O(k)$.

Já para o TAD Cláusula, essa exceção provém da função `Cláusula::acharPassagensQueSatisfacam`. Esse método realiza uma busca em uma árvore. Dessa forma, como descrito acima, a complexidade espacial desse procedimento é $O(n)$, por realizar pesquisa em uma árvore binária AVL.

3.2.3 Passagem

Como no caso anterior, a complexidade espacial do TAD Passagem também é muito mais simples de ser analisada que a temporal. Todas as funções dessa classe tomam espaço constante durante sua execução, uma vez que não alocam memória de forma expressiva (isto é, que dependa do tamanho da entrada). Só há um método interno à classe que foge a essa regra: a função `Passagem::ordena`. Como mencionado anteriormente, esse método é a implementação de um quicksort para ordenar as passagens aéreas com base nos critérios de ordenação de cada consulta. Assim sendo, esse método toma entre $O(logn)$ (melhor caso e caso médio) e $O(n)$ (pior caso) de espaço auxiliar.

3.2.4 Lista e Célula

Todos os métodos desses TADs são constantes em espaço. Ou seja, nenhum deles requer alocação de memória extra para serem executados. No entanto, a lista em si (ou seja, o objeto da lista, com os ponteiros para os dados e para o próximo nodo) requer um espaço de $O(n)$, sendo n o número de elementos da lista. O TAD Célula é absolutamente constante em espaço, $O(1)$, pois cada célula armazena um número fixo (no caso, 1) de elementos.

3.2.5 Função main

Diferentemente da complexidade temporal, a complexidade espacial da função main é a soma de suas "subcomplexidades". Dessa forma, tem-se: $O(n) + O(k) + 8O(n) + kO(logn) = O(n)$ (respectivamente: vetor de passagens, vetor de consultas, vetor de árvores, k chamadas ao quicksort). Portanto, a função main é linear (no número de passagens aéreas) em quantidade de espaço extra necessária.

4 Estratégias de robustez

Ao longo do desenvolvimento deste trabalho, foi realizada a implementação de blocos de try-catch em diversas partes do código, como uma das principais estratégias de programação defensiva adotadas para garantir a robustez do sistema. Esses blocos desempenharam um papel fundamental ao permitir a detecção e o tratamento de exceções que poderiam ocorrer durante a execução do programa. A utilização dessa abordagem foi crucial para evitar falhas inesperadas que poderiam comprometer não apenas o funcionamento de componentes individuais, mas também a estabilidade geral da aplicação. Além de prevenir a interrupção abrupta da execução, os blocos de try-catch permitiram lidar com situações inesperadas de forma controlada, reduzindo os riscos associados a erros não tratados. Essa prática foi essencial para conferir maior confiabilidade ao sistema e para promover um desenvolvimento pautado na qualidade e na prevenção de problemas.

5 Análise experimental

Todos os testes abaixo detalhados foram feitos numa máquina com as especificações que se seguem. Processador: Intel i7, 7ª geração, 8 cores. RAM: 16 GB. Armazenamento: 256GB SSD, 1TB HD. SO: Linux (Manjaro 24.0.4).

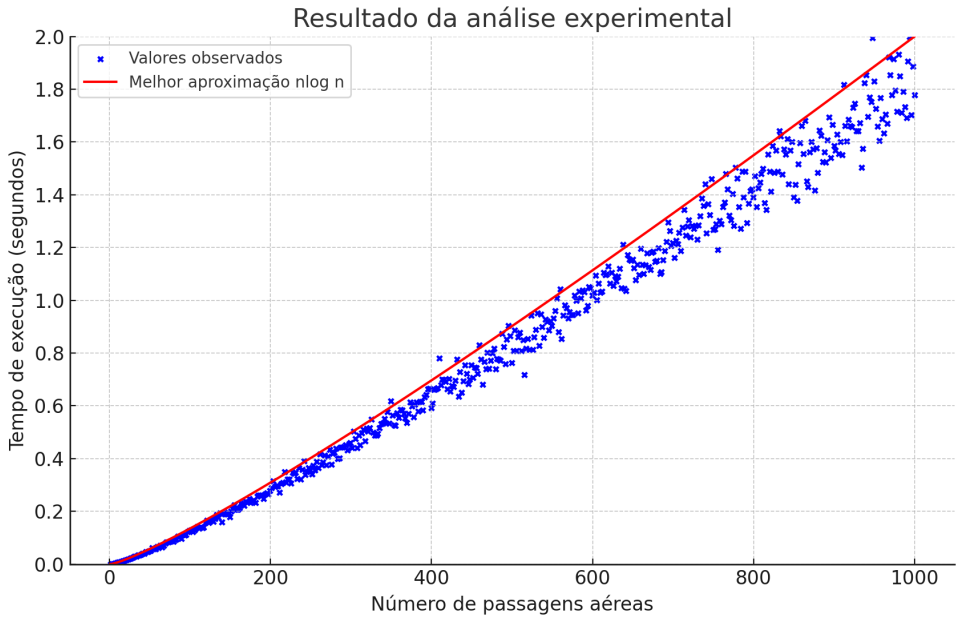


Figure 1: Os pontos azuis representam os "Valores observados" para o tempo de execução (em segundos) conforme o número de passagens aéreas varia de 1 a 1000. A curva vermelha é a "Melhor aproximação nlog n", destacando a relação esperada entre o número de passagens e o tempo de execução.

A análise experimental revela que, de fato, a complexidade geral de tempo do programa se assemelha a uma tendência observada nos testes realizados. Esse teste foi realizado com um número fixo de consultas (5% do número de passagens aéreas). Quando a quantidade de passagens aéreas começa a crescer, fica evidente a vantagem do uso da árvore AVL para a realização da pesquisa. Caso houvesse sido implementada uma busca exaustiva para encontrar as passagens que atendiam às consultas, o gráfico seria muito mais próximo de um n^2. Dessa forma, a utilização de uma estrutura balanceada como a AVL reduz significativamente o tempo de execução, garantindo um desempenho mais eficiente, especialmente para grandes volumes de dados. Esse resultado confirma a importância da escolha adequada de estruturas de dados para otimizar operações de busca, tornando o sistema escalável e viável para cenários de alta demanda.

6 Conclusões

Este trabalho explorou a implementação de um sistema eficiente de busca de passagens aéreas, utilizando diversas estruturas de dados e estratégias de ordenação. A abordagem baseada em árvores AVL permitiu uma busca eficiente, garantindo um desempenho sublinear na recuperação de passagens. Além disso, a implementação de um sistema robusto com tratamento de exceções reforça a estabilidade do código e evita falhas inesperadas. A análise experimental demonstrou a eficiência das estratégias adotadas, validando as decisões de implementação tomadas ao longo do projeto.

7 Bibliografia

Meira, W. (2024). Slides virtuais da disciplina de estruturas de dados. Disponibilizado via Moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.